Module 5: Semantic Matching, Confidence and Ranking

How a system picks the exact fields that answer each part of the question, checks the result, and ranks its options

Learning objectives

By the end of this module, you should be able to:

1. Match question slots to data fields with **maximum-weight bipartite matching**, and explain why a greedy choice can fail.
2. **Validate literal values** against the data with exact, normalized and fuzzy lookup, and infer which column a value belongs to.
3. **Combine confidence** across pipeline stages, and find the weakest stage to clarify.
4. Run **sanity checks**: consistency with reference values, robust outlier scores, and Bayesian updates after a red flag.
5. Explain **two-stage retrieval** and the trade-off between recall and precision.
6. Train a ranker with **pointwise, pairwise and listwise** objectives.
7. Add **diversity** to a ranking with MMR.
8. **Evaluate** rankings with Precision@k, Recall@k, MRR and nDCG.

5.1 The problem: from "what" to "exactly which field"

By the end of Module 4 the question has been fully parsed:

```
template(rank) = ⟨ measure = Revenue, dimension = Store, order = desc, N = 5,
                   location = Paris (France), time = [2026-07-01, 2026-10-01) ⟩
```

The chosen sources still contain dozens of columns. Three gaps remain:

Gap	Question	Sections
Matching	Which column is "Revenue"? Which is "Store"? Which date column?	5.2–5.3
Confidence	How sure are we, end to end? Does the result look plausible?	5.4–5.5
Ranking	Of all candidate sources and plans, which come first?	5.6–5.9

5.2 Matching slots to fields: bipartite matching

The setup

Build a **bipartite graph**:

- **Left nodes:** the template slots, written $S = \{s_1, \ldots, s_m\}$

- **Right nodes:** the candidate fields, written $F = \{f_1, \ldots, f_n\}$
- **Edge weights:** $w(s, f)$ = how well field f represents slot s

Weights come from the Module 2 signals: embedding similarity between names and descriptions, the lexicon, type compatibility (a measure needs a numeric field, a time slot needs a date field) and curated metadata.

The objective

Choose a **matching** M: a set of slot–field pairs in which no slot and no field is used twice.

$$M^* = \operatorname{argmax}_M \sum_{(s,f) \in M} w(s, f)$$

If each weight is a **log probability**, $w(s, f) = \log P(f \text{ represents } s)$, then maximizing the sum is the same as maximizing the **joint probability** that all the matches are right. This is the same trick used for join paths in Module 3.

Why greedy fails

Greedy matching takes the single best pair first, then the best remaining pair, and so on. Here is a small case where that goes wrong:

	`amount`	`sales_total`
Revenue	0.90	0.80
Refund	0.85	0.10

`amount` is a generic name, so it fits both concepts. `sales_total` only fits Revenue.

```
Greedy:   Revenue → amount (0.90), then Refund → sales_total (0.10)    total = 1.00
Optimal:  Revenue → sales_total (0.80),  Refund → amount (0.85)        total = 1.65
```

Greedy grabbed the most attractive pair and left Refund with a nonsense match. The optimal solution gives up a little on Revenue to gain a lot on Refund.

The Hungarian algorithm (intuition)

The **Hungarian algorithm** finds the optimal matching in $O(n^3)$ time. The idea:

1. Turn it into a cost-minimization problem (cost = max weight - weight).
2. Subtract each row's minimum, then each column's minimum, so that zeros mark "best available" choices.
3. Look for a full set of zeros in which no two share a row or column. If none exists, adjust the costs and repeat.

The result is the globally best assignment, never a locally greedy one.

Practical variations

Variation	When it applies
Unmatched slot	No field has $w \geq \tau$, so leave the slot empty and fill it with a prior, or ask (Module 3, section 3.9)

One field serving two slots	The same column can be both a dimension and a filter ("stores *in* Paris, *by* store")
Type hard constraints	w(s, f) = −∞ when the types are incompatible (a sum over a text column)

5.3 Validating values against the data

A literal value in the question ("EMEA", "California", "Gold") must actually **exist** in the matched field. Otherwise the query returns nothing, or the wrong rows.

Three levels of lookup

Level	Rule	Example
Exact	v ∈ V(f)	"EMEA" = "EMEA"
Normalized	norm(v) ∈ norm(V(f)), ignoring case, spaces and punctuation	"emea " → "EMEA"
Fuzzy	max_{u ∈ V(f)} sim(v, u) ≥ τ_v	"Calfornia" → "California"

Fuzzy example: "Calfornia" vs. "California" is one insertion apart, and the longer word has 10 letters.

```
sim = 1 − 1/10 = 0.90   ≥ τ_v = 0.85   ⇒  accept, and mention the correction
```

Which column holds the value?

Sometimes the value is known but the field isn't. *"Revenue from Gold customers"*: where does "Gold" live?

Treat it as Bayesian inference over the candidate fields:

```
P(f | v) ∝ P(v | f) · P(f | slot)
```

- P(v | f) is how common v is in field f.
- P(f | slot) is how well the field's type fits the slot. The template expects a *customer* attribute.

Field	Rows with "Gold"	P(v | f)	P(f | customer slot)	Product
`customer_tier`	1,200 of 10,000	0.12	0.8	**0.096**
`product_color`	30 of 50,000	0.0006	0.1	**0.00006**

The ratio is 0.096 / 0.00006 = **1,600 : 1** in favour of `customer_tier`.

The empty-result check

A query that returns **zero rows** is strong evidence that a value or filter is wrong, rather than evidence that the true answer is zero:

`if |result| = 0:  re-check each filter value → relax filters one at a time → find which one empties the result`

5.4 Combining confidence across stages

Every stage so far produced a probability. If the stages are roughly **independent**, the probability that **everything** is right is their product:

$$Conf = \Pi_i \; p_i \qquad \log Conf = \Sigma_i \log p_i$$

Worked example

Stage	Module	p_i
Intent (rank)	4	0.90
Source choice	1	0.66
Join path	3	0.74
Entity (Paris, France)	4	0.69
Field matching	5	0.85

$$Conf = 0.90 \times 0.66 \times 0.74 \times 0.69 \times 0.85 \approx 0.258$$

Every stage looks reasonable on its own, yet the overall confidence is only about 26%. That isn't a flaw in the math: it honestly reflects that five uncertain choices compound. It is also why borderline choices should be **stated as assumptions**.

> **Per-stage average.** To compare plans with different numbers of stages, the geometric mean is often reported: Conf^(1/5) = e^(ln 0.258 / 5) ≈ **0.76**. It shows "typical stage quality" without penalizing long pipelines.

Which stage should we clarify?

Asking the user to settle a stage raises that stage's p_i to 1. The new overall confidence becomes Conf / p_i, so **the lowest p_i gives the largest gain**.

Settle this stage	New Conf
Source choice (0.66)	0.258 / 0.66 ≈ **0.391** ← largest gain
Entity (0.69)	0.258 / 0.69 ≈ 0.374
Join path (0.74)	0.258 / 0.74 ≈ 0.349
Intent (0.90)	0.258 / 0.90 ≈ 0.287

> **Rule:** ask about the **weakest link**. One well-chosen clarifying question beats several vague ones.

When stages are not independent

If two stages depend on each other (for example, the entity choice makes a particular source more likely), the simple product **under-counts** confidence. The general form is the chain rule:

$$P(\text{all correct}) = P(s_1) \cdot P(s_2 \mid s_1) \cdot P(s_3 \mid s_1, s_2) \cdot \ldots$$

5.5 Sanity checks: does the answer look plausible?

Even a well-built plan can produce an implausible number, often because of a join trap (Module 3, section 3.7) or a wrong filter. Sanity checks compare the answer with **what we'd expect**.

Check 1: Consistency with a reference value

If an independent reference A_ref exists (a curated report, a different source), measure agreement:

$$\text{Consistency}(A) = \exp\left(-\eta \cdot \frac{|A - A_{ref}|}{|A_{ref}|} \right)$$

Example: A = \$1.05M, A_ref = \$1.00M and η = 2. The relative gap is 5%.

$$\text{Consistency} = \exp(-2 \times 0.05) = e^{-0.1} \approx 0.905$$

A 50% gap gives e^(-1) ≈ 0.37: a strong signal that something differs, such as the definition, the filters or the time window.

Check 2: Robust outlier score

Compare the result with the history of the same metric. Plain z-scores are distorted by past outliers, so use the **median** and the **median absolute deviation (MAD)**:

$$\text{modified } z = 0.6745 \cdot \frac{x - \text{median}}{\text{MAD}} \qquad \text{flag if } |\text{modified } z| > 3.5$$

Example: monthly revenue has a historical median of \$1.00M and a MAD of \$0.08M. The new result is **\$3.10M**.

$$\text{modified } z = 0.6745 \times \frac{3.10 - 1.00}{0.08} = 0.6745 \times 26.25 \approx 17.7 \quad \Rightarrow \quad \text{flag}$$

A value of about 3× normal is the classic signature of a **fan-out join** (Module 3).

Check 3: Structural invariants

These are rules that must always hold:

Invariant	Violation suggests
COUNT(DISTINCT customers) ≤ total customers	Duplicated rows
Shares of a whole sum to ≈ 100%	Overlapping categories, or missing ones
Non-negative measures stay ≥ 0	Sign errors, refunds mixed in
Sub-totals add up to the total	Inconsistent filters across parts

A rate stays between 0 and 1	Numerator and denominator at different grains

Updating confidence after a red flag

A failed check is **evidence**. Use Bayes' rule to update the prior confidence:

```
P(correct | flag) = P(flag | correct) · P(correct) / P(flag)
```

Example: prior P(correct) = 0.8. Correct answers rarely trigger the flag, so P(flag | correct) = 0.02. Wrong answers often do, so P(flag | wrong) = 0.5.

```
P(flag) = 0.02 × 0.8 + 0.5 × 0.2 = 0.016 + 0.100 = 0.116
P(correct | flag) = 0.016 / 0.116 ≈ 0.14
```

Confidence drops from **80% to 14%**. The right response is to re-examine the plan, not to report the number.

5.6 Ranking and retrieval: two stages

There may be **thousands** of candidate sources or plans. Scoring each one in depth is too slow, so rankers work in two stages:

```
Stage 1 (recall):     cheap score over everything      →  keep the top K   (e.g. K = 50)
Stage 2 (precision):  rich score over those K only     →  final order
```

Stage 1: retrieve broadly

- Embedding nearest-neighbour search with cos(φ(Q), φ(d)) (Module 1), plus a keyword score.
- **Approximate** nearest-neighbour methods trade a little accuracy for very large speed-ups.
- The goal is **not to miss** anything relevant. It's measured by recall at K:

```
Recall@K = | relevant items in top K | / | all relevant items |
```

If Stage 1 misses a relevant item, Stage 2 can never recover it. Stage 1 errors are **permanent**.

Stage 2: re-rank precisely

Score each of the K survivors with a richer function of many features:

```
r(Q, d) = f_θ( cos, BM25, concept coverage, prior P(d), field-match quality, value-validation pass, sanity score, … )
```

The simplest form is linear, $r = \Sigma_j \theta_j \cdot x_j$. More expressive forms (trees, neural networks) capture interactions between features.

The trade-off

K	Effect
Small	Fast, but Recall@K falls and good candidates are lost
Large	Slow, but Stage 2 sees more good candidates

Choose K as the smallest value at which Recall@K reaches a target, such as 95%.

5.7 Learning to rank

How do we choose the weights θ? We learn them from examples of good and bad rankings. There are three families of objectives.

Pointwise: predict each item's relevance

```
ℒ = Σ_d ( r(Q, d) − label(d) )²        or cross-entropy for yes/no labels
```

This is simple, but it optimizes absolute scores, while ranking only cares about **order**.

Pairwise: get pairs in the right order

For each pair (d^+ more relevant than d^-), penalize the ranker when d^- scores close to or above d^+:

```
ℒ_pair = log( 1 + exp( −( r(d⁺) − r(d⁻) ) ) )
```

Scores	Gap $r(d^+) - r(d^-)$	Loss
$r(d^+) = 2$, $r(d^-) = 1$ (correct order)	+1	$\ln(1 + e^{-1}) \approx$ **0.313**
$r(d^+) = 1$, $r(d^-) = 1$ (tied)	0	$\ln 2 \approx$ **0.693**
$r(d^+) = 1$, $r(d^-) = 2$ (wrong order)	−1	$\ln(1 + e^{1}) \approx$ **1.313**

The loss shrinks smoothly as the correct item pulls ahead, and grows as the order flips.

Listwise: optimize the whole list

Treat the scores as a probability distribution over which item belongs at the top, and match it to the ideal distribution:

```
P_θ(d at top) = softmax_d( r(Q, d) )
ℒ_list = − Σ_d P_ideal(d) · log P_θ(d at top)
```

Listwise methods most directly target metrics like nDCG (section 5.9), because they weight mistakes near the top more heavily.

Family	Optimizes	Strength	Weakness
Pointwise	Each score	Simple	Ignores order
Pairwise	Pair order	Robust and intuitive	Treats all pairs equally
Listwise	Whole list	Matches ranking metrics	More complex

5.8 Diversity: maximal marginal relevance (MMR)

A ranking full of **near-duplicates** wastes the top positions. Three copies of the same table rank highly, but tell the user nothing new.

The MMR rule

Build the list one item at a time. Each pick balances relevance against similarity to what has already been chosen:

```
next = argmax_{d ∉ S} [ λ · r(Q, d)  −  (1 − λ) · max_{s ∈ S} sim(d, s) ]
```

- λ = 1: pure relevance (no diversity)
- λ = 0: pure novelty (ignores relevance)

Worked example (λ = 0.7)

Candidate	r(Q, d)	sim to d_1
d_1 certified summary table	0.90	—
d_2 copy of d_1 in another workspace	0.88	0.95
d_3 detailed transaction table	0.70	0.20

Pick 1: nothing is selected yet, so choose the most relevant: **d_1**.

Pick 2:

```
MMR(d₂) = 0.7 × 0.88 − 0.3 × 0.95 = 0.616 − 0.285 = 0.331
MMR(d₃) = 0.7 × 0.70 − 0.3 × 0.20 = 0.490 − 0.060 = 0.430   ← chosen
```

d_3 ranks second, ahead of the near-duplicate d_2, even though d_2 is more relevant. The user now sees two genuinely different options: a summary and its detail.

5.9 Evaluating rankings

How do we know a ranker is good? Compare its output to human relevance judgments.

Precision and recall at k

```
Precision@k = relevant items in top k / k
Recall@k    = relevant items in top k / all relevant items
```

These metrics ignore order **within** the top k.

Mean reciprocal rank (MRR)

This measures how high the **first** relevant item appears, averaged over queries:

```
MRR = (1 / |queries|) · Σ_q  1 / rank of first relevant item for q
```

Example: across three queries, the first relevant item appears at ranks 1, 3 and 2.

```
MRR = (1/1 + 1/3 + 1/2) / 3 = 1.833 / 3 ≈ 0.611
```

Normalized discounted cumulative gain (nDCG)

nDCG uses **graded** relevance (0 = irrelevant … 3 = perfect) and **discounts** items lower in the list:

```
DCG@k  = Σ_{i=1}^{k} (2^{rel_i} − 1) / log₂(i + 1)
nDCG@k = DCG@k / IDCG@k          (IDCG = DCG of the ideal ordering; nDCG ∈ [0, 1])
```

Example: the system returns relevance grades **[2, 3, 0]**. The ideal order is **[3, 2, 0]**.

$$\text{DCG} = (2^2 - 1)/\log_2 2 + (2^3 - 1)/\log_2 3 + 0 = 3/1 + 7/1.585 \approx 3.000 + 4.417 = 7.417$$
$$\text{IDCG} = (2^3 - 1)/\log_2 2 + (2^2 - 1)/\log_2 3 + 0 = 7/1 + 3/1.585 \approx 7.000 + 1.893 = 8.893$$

$$\text{nDCG@3} = 7.417 / 8.893 \approx 0.834$$

Putting the second-best item first costs about **17%** of the ideal score. The exponential gain (2^rel - 1) means that **misplacing the best item hurts most**.

Metric	Answers	Uses grades?	Order-aware?
Precision@k	How much of the top k is relevant?	No	No
Recall@k	How much of the relevant set did we find?	No	No
MRR	How soon do we hit the first good item?	No	First hit only
nDCG@k	How close is the order to ideal?	Yes	Yes

5.10 Putting it together: the candidate score

For each candidate plan π (sources, joins, field matches, filters), combine everything from this module into one score:

$$\log \text{Score}(\pi) = \sum_{(s,f) \in M^*} \log P(f \text{ represents } s) \quad \leftarrow \text{field matching (5.2)}$$
$$+ \sum_{\text{values}} \log P(\text{value valid in field}) \quad \leftarrow \text{value validation (5.3)}$$
$$+ \sum_{\text{stages}} \log p_i \quad \leftarrow \text{stage confidence (5.4)}$$
$$+ \log \text{Consistency}(A_\pi) + \log P(\text{correct} \mid \text{checks}) \quad \leftarrow \text{sanity checks (5.5)}$$
$$+ \log r(Q, \text{sources of } \pi) \quad \leftarrow \text{retrieval rank (5.6–5.7)}$$

The candidates are then **ordered by score** and **diversified** with MMR (5.8). The leading candidates go to Module 6, which selects the final answer.

5.11 Common pitfalls

Pitfall	Symptom	Fix in the math

Greedy field matching	One slot gets a nonsense field	Hungarian / global maximum matching
Ignoring field types	Summing a text column, filtering a number with text	w = -∞ for incompatible types
No value validation	Empty results, or silently wrong rows	Exact → normalized → fuzzy lookup
Reading each stage's confidence alone	Overconfident overall answer	Multiply stages; clarify the weakest link
Skipping sanity checks	Reporting a 3× fan-out number	Robust z-scores, invariants, Bayesian update
Stage 1 K too small	Best source never considered	Choose K by a Recall@K target
Pointwise-only training	Good scores, poor ordering	Pairwise or listwise objectives
Near-duplicates at the top	Same table listed three times	MMR diversity
Evaluating only by precision	Good items buried at rank 5	nDCG / MRR (order-aware)

Check your understanding

1. **Compute:** w(A, x) = 0.6, w(A, y) = 0.5, w(B, x) = 0.55 and w(B, y) = 0.2. What totals do greedy and optimal matching give?
2. **Compute:** three stages have confidences 0.9, 0.8 and 0.5. What is the overall confidence, and what does it become after clarifying the weakest stage?
3. **Compute:** what is the pairwise loss when $r(d^+) = r(d^-)$?
4. **Compute:** the relevance grades in rank order are [0, 3]. What is nDCG@2?
5. **Compute:** for two queries, the first relevant item appears at rank 2 and rank 4. What is the MRR?
6. **Reason:** why might each stage look reliable while the overall answer is still uncertain?

<details>
<summary>Answers</summary>

1. Greedy: A → x (0.6), then B → y (0.2), giving **0.80**. Optimal: A → y (0.5) and B → x (0.55), giving **1.05**.
2. 0.9 × 0.8 × 0.5 = **0.36**. Clarifying the 0.5 stage gives 0.36 / 0.5 = **0.72**.
3. $\ln(1 + e^0) = \ln 2 \approx$ **0.693**.
4. DCG = 0 + 7 / $\log_2 3 \approx 4.417$. IDCG = 7 / 1 + 0 = 7. nDCG = 4.417 / 7 ≈ **0.631**.
5. (1/2 + 1/4) / 2 = **0.375**.
6. Confidence **multiplies** across stages. Five stages that are each about 75–90% reliable can combine to well under 50%, because an error at **any** stage spoils the final answer.

</details>

Key takeaways

- **Field matching** is a global assignment problem. The Hungarian algorithm beats greedy picking, and log-probability weights turn it into joint probability.
- **Validate every value** against the data, and infer unknown columns with Bayes' rule. An empty result means a filter is wrong.
- **Confidence compounds:** multiply across stages, and ask about the **weakest link**.
- **Sanity checks** are evidence. A failed check can rightly collapse confidence through Bayes' rule.
- **Retrieval** runs in two stages: recall first (errors there are permanent), then precision.
- **Learning to rank** should optimize order, through pairwise or listwise objectives.
- **MMR** keeps near-duplicates from crowding the top, and **nDCG and MRR** judge rankings by order, not just membership.

Coming next: Module 6, Answer Selection and the Full Pipeline

With ranked, validated candidates in hand, how is the **final** answer chosen? We'll cover expected utility, hard constraints, satisficing all the user's filters at once, abstention and clarification as decisions, choosing how to present the answer, and a complete worked example that runs one question through all six modules.

Report generated by Databricks Genie